Advanced usage of Git versioning system

Git forges, automation and CI/CD

Vojtěch Trefný, Alexander Zgabur

24. 3. 2026

Homeworks

Deadlines

- **HW 3:** 30th March 2026, 23:59
- **HW 4:** 13th April 2026, 23:59

Homework 4

- Sync your fork. Open a new MR updating your `<uco>.txt` file.
- CI pipeline will fail on your MR.
- Debug the issue and update your `<uco>.txt` to comply with new linting rules.

Git Forges

Git Forge

- **Forge** *is a web-based collaborative software platform for both developing and sharing computer applications.*[1]
- Git forge is a forge where Git is used as the version control system.
- Usually offers multiple services:
  - Pull/merge requests interface, including discussions and review.
  - Issue tracker.
  - Project management tools (e.g. Kanban board).
  - Continuous integration (CI) and Continuous delivery (CD) tools and environment.
  - Release management and software hosting.
  - Wiki and/or website hosting.

[1] https://en.wikipedia.org/wiki/Forge_(software)

GitHub

- Probably the best known and most used Git forge these days.
- For many, a de-facto synonym for Git.
- Proprietary, currently owned by Microsoft.
- Public projects are hosted for free, private projects are limited for non-enterprise users.
- Offers CI/CD integration, custom command line tool (`gh`) and integration with many third party projects.

GitLab

- Open core Git forge used by many open source projects and organizations.
- Cloud-hosted `GitLab.com` is available for free for individuals and open source projects.
- Community edition (available under MIT license) can be self-hosted.
- Offers CI/CD integration, custom command line tool (`glab`) and integration with many third party projects.

Forgejo

- Open-source self-hosted Git forge.
- Includes integrated Wiki, Kanban board, CI (Forgejo actions), Pages, Translations etc.
- Projects can be migrated to Forgejo from GitHub, GitLab and others, including issues, wiki, releases etc.
- Public instance for hosting free and opensource projects is available on `Codeberg.org`.

CI/CD

What is CI/CD

Continuous Integration (CI)

- *Practice of integrating source code changes frequently and ensuring that the integrated codebase is in a workable state.*[1]
- Each change is automatically built and tested.

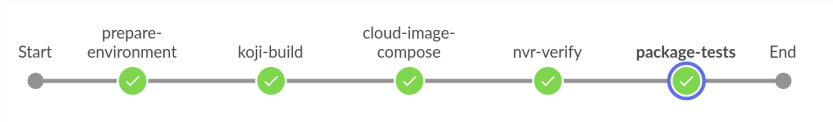
Continuous Delivery / Deployment (CD)

- **Continuous Delivery** – code changes are automatically prepared for release and manual deployment.
- **Continuous Deployment** – every change that successfully passes the pipeline is automatically deployed to production.

[1] https://en.wikipedia.org/wiki/Continuous_integration

CI/CD Pipeline

- Steps that need to be performed to test and deliver a new version of the software.
- Defines what needs to be done: when, how and in what order.
- Steps can vary for every project.
- Multiple pipelines or steps can run in parallel.



CI/CD Pipeline

1. Testing environment

Preparation of the environment to run the tests: deploying containers, starting VMs...

2. Static Analysis

Finding defects by analyzing the code without running it.

3. Code style

Checking for violations of the language or project style guides.

4. Build

Building the project from source.

5. Tests

Running project test suite or test suites.

6. Packaging and Deployment

Building source archives, packages or container images.

Testing Environment

Configuration Matrix	x86_64	i686	arm64
f_30	✓	✓	○
f_31	✓	○	✓
f_rawhide	✓	○	○
centos_7	✓	○	○
debian_10	✓	✓	○
debian_t	✓	○	○
rhel_8	✓	○	○

1. Preparation of VMs/containers to run the tests

We might want to run tests in different environments on multiple different distributions or architectures.

2. Installation of the test dependencies

Test dependencies are usually not covered by the project dependencies.

3. Getting the code

Clone the PR or get the latest code from the main branch.

Static Analysis

- Tools that can identify potential bugs by analyzing the code without running it.
- Can detect problems not covered by the test suite – corner cases, error paths etc.
  - Coverity (C/C++, Java, Python, Go. . .)[1]
  - Cppcheck (C/C++)[2]
  - Pylint (Python)[3]
  - RuboCop (Ruby)[4]

[1] https://scan.coverity.com
[2] https://cppcheck.sourceforge.io/
[3] https://www.pylint.org
[4] https://docs.rubocop.org

Coverity

```
Error: USE_AFTER_FREE (CWE-825):
libblockdev-2.13/src/plugins/lvm-dbus.c:1163: freed_arg: "g_free"
frees "output".
libblockdev-2.13/src/plugins/lvm-dbus.c:1165: pass_freed_arg: Passing freed
pointer "output" as an argument to "g_set_error".
# 1163|       g_free (output);
# 1164|       if (ret == 0) {
# 1165|->         g_set_error (error, BD_LVM_ERROR, BD_LVM_ERROR_PARSE,
# 1166|                        "Failed to parse number from output: '%s'",
# 1167|                        output);
```

Runtime Analysis

- Tools that can identify bugs during runtime.
- Needs the code to actually run, either through manual testing or when running the test suite.
  - Valgrind – memory management and threading bugs[1]
  - ASan – *AddressSanitizer* – memory management bugs (buffer overflow, dangling pointers...). Part of the LLVM project, integrated into `gcc` and `clang`[2]

[1] https://valgrind.org/
[2] https://github.com/google/sanitizers/wiki/AddressSanitizer

Code style and style guides

- Coding conventions – naming, code lay-out, comment style...
- Language specific (PEP 8[1]), project specific (Linux kernel coding style[2]) or library/toolkit specific (GTK coding style[3]).
- Automatic checks using specific tools (`pycodestyle`) or (partially) by the static analysis tools.

[1] https://www.python.org/dev/peps/pep-0008/
[2] https://www.kernel.org/doc/html/v5.11/process/coding-style.html
[3] https://developer.gnome.org/programming-guidelines/stable/c-coding-style.html.en

Build

- Building the project, a preparation to run the test suite.
- Depends on language – mostly no-op for interpreted languages, more complicated for compiled ones.
- Build in the CI environment can detect issues with dependencies.
- Builds on different architectures can help detect issues related to endianness or data type sizes.

Tests

- Running tests that are part of the project.
- New tests should be part of every change to the codebase.
  - New features require new unit and integration tests.
  - Bug fixes should come with a regression test.
- For some projects (like libraries) running test suites of their users might be an option.

Coverage

- Code coverage (or Test coverage) represents how much of the code is covered by the test suite.
- Usually a percentage value that shows how many lines of the code were "visited" by the test.
- Generally a check that all functions and branches are covered by the suite.
- Used as a measure of the test suite "quality".

Coverage

```
Name    Stmts   Miss Branch BrPart  Cover   Missing
-------------------------------------------------------------------------------
a.py      487      9    178     11    97%   206, 268, 377->376, 393->392, 418,
                                             448, 452, 460, 660, 729, 746,
                                             889->891, 891->894
b.py      220      8     74      8    95%   81, 173, 193->197, 279, 340, 342, 346
c.py       19      9      8      2    44%   35-36, 50->48, 53, 60-70
d.py        5      0      6      1    91%   31->exit
e.py       46      0      4      0   100%
...
-------------------------------------------------------------------------------
TOTAL    3600   1477   1381    100    56%
```

Packaging and publishing

- **Delivery** – releasing new changes quickly and regularly (daily, weekly...).
- **Deployment** – delivery with automated push to production, without human interaction.

- Usually after merging the changes, not for the PRs.
- Building packages, container images, ISO images...
- Built packages can be used for further testing (manually by the Quality Assurance or in another CI infrastructure) or directly pushed to production or included in testing/nightly builds of the project.

Packit

- Tool for automating packaging and testing of upstream projects for Fedora.
- Integrates with GitHub and GitLab: automatically builds RPM packages and runs tests on pull requests.

packit-as-a-service bot commented 24 days ago

Congratulations! One of the builds has completed. 🍾

You can install the built RPMs by following these steps:

- `sudo yum install -y dnf-plugins-core` on RHEL 8
- `sudo dnf install -y dnf-plugins-core` on Fedora
- `dnf copr enable packit/storaged-project-blivet-gui-157`
- And now you can install the packages.

Please note that the RPMs should be used only in a testing environment.

Homework

HW 5

Instructions

- Create your own repo on GitLab (or equivalent).
- Set up GitLab CI or GitHub actions on the repo and create at least one action that will react to a MR/PR in some meaningful way (runs *tests*, writes a comment, etc.).
- Open a MR/PR demonstrating the action.
- If you already have an existing project where this could be beneficial to you, you can use that project.
- „Odpovědník“ will be prepared where you'll fill in the URL of your repo.

Deadline

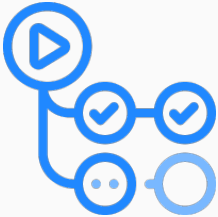
- 20th April 2026, 23:59

CI Tools

Demo

GitHub Actions

- Automation *framework* integrated into GitHub.
- Covers not only CI but also CD (publishing packages on various services and deploying on many public clouds) and project and issue management.
- Free for all public repositories, limited and paid options for private projects.
- https://github.com/features/actions



GitHub Actions – Testing Environment Setup

```
runs-on: ubuntu-latest                    <-- System to run the tests on

steps:
- uses: actions/checkout@v4               <-- Getting the code
- name: Set up Python 3.10
  uses: actions/setup-python@v3           <-- Environment configuration
  with:
    python-version: "3.10"
- name: Install dependencies
  run: |                                  <-- Dependencies installation
    python -m pip install --upgrade pip
    pip install flake8 pytest
    if [ -f requirements.txt ]; then pip install -r requirements.txt; fi
```

GitHub Actions – Static Analysis and Tests

```
    steps:
...
    - name: Lint with flake8            <-- Static analysis
      run: |
        # stop the build if there are Python syntax errors or undefined names
        flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics
        # exit-zero treats all errors as warnings
        flake8 . --count --exit-zero --max-complexity=10 --max-line-length=127
    - name: Test with pytest            <-- Tests
      run: |
        pytest
```

GitHub Actions Demo

Repository with examples used during the demo:

https://github.com/vojtechtrefny/github-actions-examples

GitLab CI

- CI/CD automation integrated into GitLab.
- Configuration is done with a YAML file in the repository.
- Pipelines/tests can run either on infrastructure provided by GitLab or on custom runners.
- https://docs.gitlab.com/ee/ci/

GitLab CI

4 jobs for `main` in 1 minute and 2 seconds

latest

e5546e84

No related merge requests found.

Pipeline Needs Jobs 4 Tests 0

Build	Test	Deploy
build-job	test-job1	deploy-prod
	test-job2	

Continuous Integration @ GitLab FI

- In *Settings* → *General* → *Permissions*, enable the *Pipelines* option if it is not already enabled.
- In the `.gitlab-ci.yml` settings, add the `.shared-fi` tag to each target, e.g.:

```
build:
  tags:
    - shared-fi
```

- More information about GitLab CI @ FI MUNI is available at https://www.fi.muni.cz/tech/unix/gitlab/ci.html

Jenkins

- Automation system, not a "true" CI/CD tool.
- Can automatically run given tasks on a node or set of nodes.
- Tasks can be started on a time basis or triggered by an external event (like a new commit or PR on GitHub).
- https://jenkins.io/